Nakijk 03_02 — V1

NAKIJK V1 · Hoofdstuk 03 · CRUD via een repository · bij Oefening_03_02.md · 7 november 2026

Nagekeken: je project in `Oefeningen Hoofdstuk 3`, zoals het om 21:01 bewaard was. Ik kon je API hier niet starten, want er is geen .NET: ik heb je code gelezen en elk verzoek uit de opgave op papier nagelopen.

Legende: ✓ in orde · ✗ fout of afwijking, met de plaats waar je het oplost · ○ nog niet gemaakt

Kort oordeel

De refactoring zit goed in elkaar. Je lijst staat in de repository en overleeft een volgend verzoek, je controller beslist en de repository voert uit, en alles komt via de constructor binnen. Eén fout breekt een ventilator bij elke PUT (A1). De rest van punt 1 tot 5 zijn kleinigheden, en punt 6 is begonnen.

Volgorde: eerst A1, dan B1 tot B3, dan het vervangen, en daarna de historiek. Lees blok V voor je aan de historiek verdergaat: daar staan vier dingen die je anders pas merkt als alles samenkomt.

Takenlijst

1. A1 — bijwerken zet de Id uit de body (`InMemoryVentilatorRepository`, regel 38)
2. B1 — POST, PUT en DELETE loggen geen binnenkomend verzoek
3. B2 — de reden in drie Warnings klopt niet met de regel
4. B3 — `Replace` uit het contract halen
5. Nog te doen — vervangen: PUT `api/ventilator/vervang/{id}`
6. Nog te doen — de historiek: lees eerst V1 tot V4

Stand van zaken

Punt	Wat de opgave vraagt	Status
1	de zes endpoints antwoorden zoals in 02_02	✓ GET, GET op Id, POST, DELETE ✗ PUT: A1 ○ vervangen ontbreekt
2	het contract `IVentilatorRepository`	✓ in orde, met één methode te veel: B3
3	de repository, die zelf de Id bepaalt	✓ toevoegen ✗ bijwerken: A1
4	wie beslist wat, de constructor, Program.cs en de logging	✓ beslissingen en injectie ✗ logging: B1 en B2
5	gegevens tussen twee verzoeken	✓ in orde
6	de wijzigingshistoriek	○ begonnen: het model ✓, de repository half, de rest nog te doen. Lees V.

Zo antwoordt je API nu, vanaf een herstart:

Verzoek	Je API antwoordt nu	De opgave vraagt
GET `api/ventilator`	200 met je vier ventilatoren	✓
GET `api/ventilator/99`	404 met *We vonden geen ventilator met Id 99.*	✓
POST `api/ventilator`, geldig	201 met Id 5, en een adres	✓
POST `api/ventilator`, 92 mm	400 met *We verkopen enkel ventilatoren van 120 of 140 mm.*	✓ (de tekst in je log: B2)
PUT `api/ventilator/2`, body zonder `id`	204, maar ventilator 2 heeft nu Id 0	204, en ventilator 2 houdt Id 2 (A1)
GET `api/ventilator/2`, daarna	404	200 met de nieuwe gegevens
PUT `api/ventilator/99`	404 met het bericht uit de opgave	✓
DELETE `api/ventilator/5`, twee keer	204, daarna 404 met het bericht	✓
PUT `api/ventilator/vervang/3`	404 zonder body: er is geen endpoint	201 met de nieuwe ventilator ○
GET `api/ventilator/historiek`	400: komt bij GET `{id}` terecht (V4)	200 met `[]` ○

Wat goed zit, en waarom

Je lijst is `static` (regel 5 van je repository). Daardoor werkt punt 5. Met `AddScoped` maakt ASP.NET Core voor elk HTTP-verzoek een nieuwe controller en een nieuwe repository. Een gewoon veld hoort bij dat ene object en verdwijnt na het verzoek. Een `static` veld hoort bij de klasse zelf: er is er één voor de hele levensduur van je API, tot je hem herstart. Het is dezelfde oplossing als bij de lijst in de controller van les 02.

De controller beslist, de repository voert uit. De drie controles, de controle of een ventilator bestaat en de keuze van de statuscode staan in je controller. Je repository weigert niets: `Update` en `Delete` slaan gewoon over wat niet bestaat. Dat is precies de taakverdeling uit les 03 (Separation of Concerns).

`CreatedAtAction` met het object dat de repository teruggeeft. De header `Location` krijgt zo de Id die de repository gekozen heeft. ASP.NET Core bouwt dat adres uit de route van `GetVentilatorById`, `api/[controller]/{id}`, en vult `{id}` in met de waarde die je meegeeft: `/api/Ventilator/5`.

Injectie via de constructor, met `private readonly` velden met een underscore, en de registratie in Program.cs vóór `builder.Build()`. `readonly` betekent: na de constructor kan niemand dat veld nog naar een ander object laten wijzen.

De berichten bij elke 400 en 404 zijn letterlijk die van de opgave, en bij een PUT kijk je eerst of de ventilator bestaat en pas daarna naar de controles. Het model `Wijziging` heeft precies de drie gegevens uit de tabel van punt 6.

A. Wat een verkeerd antwoord geeft

A1. Bijwerken zet de Id uit de body ✗

Waar: `InMemoryVentilatorRepository.cs`, regel 38.

Je controller zoekt de ventilator met de Id uit de URL. Je repository vindt hem, en kopieert daarna ook de Id uit de body. Wat er dan gebeurt, hangt af van wat de client stuurt:

De client stuurt	Wat er met de ventilator gebeurt
een body zonder `id`, zoals het nakijkscript	Id wordt 0: een `int` die niet in de JSON staat, krijgt zijn standaardwaarde
het voorbeeld van Scalar, met `"id": 0`	hetzelfde: Id wordt 0
`"id": 3` bij PUT `api/ventilator/5`	twee ventilatoren met Id 3; GET `api/ventilator/3` toont enkel de eerste
dezelfde Id als in de URL	geen probleem. Daarom lijkt het te werken als je zo test.

Na zo'n PUT is de ventilator onbereikbaar op zijn eigen adres: GET `api/ventilator/2` geeft 404, en in de volledige lijst staat een ventilator met Id 0.

Waarom het fout is. De URL zegt welke ventilator je bijwerkt; de Id is geen gegeven dat verandert. In les 02 kopieert de PUT de properties één voor één, behalve de Id. De opgave zegt het ook in punt 6: bij bijwerken hoort bij de wijziging *het Id uit de URL*.

Richting. Overloop de properties van `Ventilator` en vraag je bij elke property af: is dit iets wat de client mag aanpassen, of is dit wat de ventilator identificeert? Enkel de eerste groep hoort in `Update`.

Hoe je weet dat het opgelost is. Doe PUT `api/ventilator/2` met een body zonder `id`: je krijgt 204. GET `api/ventilator/2` geeft daarna 200, met Id 2 en de nieuwe gegevens, en GET `api/ventilator` toont nog altijd vier ventilatoren, zonder Id 0.

B. Werkt, maar wijkt af van de opgave

B1. POST, PUT en DELETE loggen geen binnenkomend verzoek ✗

Waar: `CreateVentilator` (regel 41), `UpdateVentilator` (regel 67) en `DeleteVentilator` (regel 99).

Punt 4 vraagt *elk binnenkomend verzoek op niveau Information*. Je twee GET's doen dat, de andere drie niet. Ze loggen enkel als er iets misloopt.

Richting. Zet het Information-bericht als eerste regel van het endpoint, vóór de controles. Dan staat ook een verzoek dat geweigerd wordt, als binnengekomen in je log.

Hoe je het controleert. Doe een POST met 92 mm en kijk in je terminal. Je ziet eerst een regel die begint met `info:` en daarna een met `warn:`, telkens gevolgd door `ProgrammingAdvancedOefeningen.Controllers.VentilatorController`. Die naam komt van `ILogger<VentilatorController>`: zo weet je van welke klasse het bericht komt.

B2. De reden in drie Warnings klopt niet met de regel ✗

Waar: regel 51, 57 en 89.

Regel	Je log zegt	Wat er echt geweigerd wordt
51	*mag niet leeg, kleiner dan 120 of groter dan 140 zijn*	alles wat niet precies 120 of 140 is, ook 130. Een `int` kan bovendien niet leeg zijn.
57 en 89	*niet negatief of onder nul / niet onder 0 gaan*	0 of lager: een prijs van precies 0 wordt ook geweigerd

Punt 4 vraagt een Warning *met de reden in het bericht*. Wie je log later leest, moet uit die reden kunnen afleiden wat er misliep. Een reden die niet klopt, stuurt hem de verkeerde kant op.

Richting. Beschrijf de regel die echt faalde. Het eenvoudigst is dezelfde zin als die de client krijgt. Regel 83 zit al dicht bij de regel.

Hoe je het controleert. POST een ventilator van 130 mm, en een met prijs 0. Lees de twee `warn:`-regels: kloppen ze met wat je deed?

B3. `Replace` staat in het contract, maar doet niets ✗

Waar: `IVentilatorRepository.cs`, regel 10, en `InMemoryVentilatorRepository.cs`, regel 56 tot 59.

Punt 2 somt vijf dingen op die je repository moet kunnen. Vervangen hoort er niet bij: dat is verwijderen en toevoegen na elkaar. Die twee stappen combineert de controller, want tussen de twee moet hij controleren, en in punt 6 schrijft hij er twee wijzigingen bij: `verwijderd` en `toegevoegd`.

`Replace(int id)` kan bovendien niet werken: hij krijgt de nieuwe ventilator niet binnen. En een lege methode compileert, en doet stil niets. Roept iemand ze later op, dan lijkt alles gelukt.

Richting. Haal `Replace` weg uit de interface en uit de klasse. Bouw het vervangen in de controller met de methodes die je al hebt.

Nog te doen: vervangen

PUT `api/ventilator/vervang/{id}` bestaat nog niet. Daarom geeft dat verzoek nu een 404 zonder body: ASP.NET Core vindt geen enkel endpoint met vier stukken in de route.

Hoe de route samengesteld wordt. De route van je controller, `api/[controller]`, komt vóór de route van elk endpoint. `[controller]` wordt de naam van je klasse zonder *Controller*: `Ventilator`. Geef je het endpoint de route `vervang/{id}`, dan wordt dat samen `api/Ventilator/vervang/{id}`. Hoofdletters spelen bij een route geen rol.

Wat de opgave vraagt: 201 met de nieuwe ventilator en zijn adres, 404 met *Kan ventilator met Id {id} niet vervangen, omdat deze niet gevonden is.*, of 400 met een van de drie redenen. Eerst kijk je of de oude ventilator bestaat, pas daarna of de nieuwe door de drie controles geraakt.

Richting. Alles wat je nodig hebt, staat al in je repository: opzoeken, verwijderen en toevoegen. Het adres in de 201 wijst naar de nieuwe ventilator, zoals bij je POST.

V. Voor je verder gaat met de historiek

V1. Je lijst met wijzigingen leeft niet lang genoeg ✗

Waar: `InMemoryWijzigingRepository.cs`, regel 5.

Deze lijst is niet `static`. Met `AddScoped` krijgt elk verzoek een nieuwe repository, en dus een nieuwe, lege lijst. Je historiek zou altijd leeg zijn. Dat is de laatste waarschuwing van de opgave: *je hebt de vraag uit punt 5 voor één van je twee repositories opgelost, maar niet voor de andere.*

Richting. Je hebt het al opgelost, in regel 5 van `InMemoryVentilatorRepository`.

V2. Program.cs kent je tweede repository nog niet ✗

Waar: `Program.cs`, na regel 14.

Zodra de constructor van `VentilatorController` om een `IWijzigingRepository` vraagt, kan ASP.NET Core de controller niet meer maken. Dan geeft **elk** verzoek naar die controller een 500, ook GET `api/ventilator`. In je terminal staat dan *Unable to resolve service for type IWijzigingRepository while attempting to activate VentilatorController*.

Richting. Regel 14 toont hoe je een repository registreert.

V3. Het contract mist bewaren, en de eerste wijziging heeft niets om op te tellen ✗

Waar: `IWijzigingRepository.cs` en `InMemoryWijzigingRepository.cs`.

Punt 6 vraagt twee dingen: een wijziging bewaren, waarbij de repository zelf de Id bepaalt, en alle wijzigingen teruggeven, de oudste eerst. Het tweede heb je ✓. Het eerste ontbreekt nog. `wijzigingById` vraagt de opgave niet.

Neem je voor de Id dezelfde aanpak als bij de ventilatoren, de hoogste Id plus één, dan loopt de allereerste wijziging vast. Je historiek begint leeg, en de hoogste waarde van een lege lijst bestaat niet: `Max` gooit dan een fout, en je API geeft een 500. Erger nog: bij een POST staat de ventilator dan al in de lijst, terwijl de client een 500 krijgt. Dezelfde fout zit trouwens in je ventilatorrepository, als iemand alle ventilatoren verwijdert en er daarna een toevoegt.

Richting. Vraag je af welke Id de eerste wijziging moet krijgen, en zoek in je leerboek, bij LINQ, wat `Max` doet met een lege lijst en hoe je voor een lege lijst een startwaarde krijgt. Er zijn ook andere manieren om het volgende nummer te kennen: kies er zelf een.

Over de naam `wijzigingById`. In C# begint een methode met een hoofdletter. Visual Studio toont daarom drie grijze puntjes onder de naam: suggestie IDE1006, een naamregel. Heb je de methode niet nodig, haal ze dan weg.

Hoe je weet dat het werkt. Herstart je API en doe meteen een geldige POST. Je krijgt 201, en GET `api/ventilator/historiek` geeft één wijziging met Id 1, `toegevoegd`, en de Id van je nieuwe ventilator.

V4. Waar de wijziging en de historiek komen

De wijziging. Bewaar ze pas als de actie gelukt is: na alle controles, en na de actie zelf. Een 400 of 404 laat geen spoor na. Bij een vervanging komen er twee: eerst `verwijderd` met de oude Id, dan `toegevoegd` met de nieuwe.

Het endpoint `historiek`. Nu komt GET `api/ventilator/historiek` bij GET `{id}` terecht. ASP.NET Core probeert van *historiek* een `int` te maken, dat lukt niet, en door `[ApiController]` krijg je automatisch een 400. Zodra je een endpoint met de route `historiek` toevoegt, verdwijnt dat probleem: een vast stuk tekst in een route weegt zwaarder dan een parameter zoals `{id}`. ASP.NET Core kiest dan zelf het juiste endpoint.

C. Opkuis, zonder invloed op het antwoord

Waar	Wat	Richting
controller, regel 18	`[HttpGet()]` met lege haakjes	`[HttpGet]` doet hetzelfde.
controller, regel 28	*GET request received for id* is Engels, de rest van je log Nederlands	Kies één taal.
controller, regel 28, 33, 71 en 105	logberichten met `$"..."`. Visual Studio toont daar mogelijk drie grijze puntjes: suggestie CA2254. De logger bewaart een waarde liever apart, met een plaatshouder in een vaste tekst. Zo kan je later op die waarde zoeken.	Werkt zoals het is. Wil je de suggestie weg, lees dan *CA2254* op Microsoft Learn.
beide repositories, regel 5	de lijst zelf wordt nooit vervangen, enkel aangevuld. Visual Studio kan IDE0044 voorstellen: *Make field readonly*.	`readonly` zegt: dit veld wijst altijd naar dezelfde lijst. Wat erin zit, mag nog veranderen. Combineer het met V1.

Is punt 6 af, draai dan het nakijkscript met `Nakijk_03_02`. Het doorloopt het testscenario voor je, en toont per stap wat je API antwoordt.